Teach-back Plan

Workshop 1, Lesson 2 · Permissions & Autonomy

10-minute excerpt · enterprise software engineers · 40-minute evaluation block

Presenter prep only. The live session uses the supplied deck as is.

OVERVIEW

Goal and scope

- Outcome: a learner can explain why a rule and a hook are separate layers
- They can predict what deny → ask → allow does with a given command
- They can name the one file to audit: .claude/settings.local.json
- Excerpt covers slides 1, 3–7 and 9; slide 8 named only; slide 2 skipped (placeholder)
- If interrupted: cut the slide 6 walk-through first, then the live demo

RUN OF SHOW

Timing

- 0:00 Slide 1: opening frame
- 0:15 Slide 3: “CLAUDE.md asks. Permissions decide.”
- 1:30 Slide 4: deny → ask → allow; first match wins; git push --force prediction
- 3:30 Slide 5: three modes, two profiles (Cautious, Working)
- 4:45 Slide 6: read as a diff; in Working the hook is load-bearing
- 6:00 Slide 7: core + live demo (rm -rf, then sh -c wrapped form)
- 8:45 Slide 8: hands-on named only (stress settings.local.json audit)
- 9:00 Slide 9: Direct, Restrict, Verify. Close.

SLIDES 3–4

Talk track: why permissions, the three lists

- CLAUDE.md is advisory; the model can reason past it
- Permission rules are evaluated deterministically, before execution
- Check: “Is ‘never push to main’ in CLAUDE.md a control?” (No, it’s a request)
- First match decides, in order deny → ask → allow; specificity doesn’t matter
- No allow exceptions inside a deny: aws * in deny blocks aws s3 ls
- Prediction: force-push in deny, push in ask → force-push denied, plain push asks
- Bridge: allow a command broadly, intercept dangerous forms with a hook

SLIDES 5–6

Talk track: modes and profiles

- default = manual · acceptEdits = auto-accept edits · auto = classifier reviews each action
- Present as three modes, not a ladder (auto adds a classifier)
- Cautious: deny secrets + curl/wget, ask on push/commit, allow tests
- Working: acceptEdits + allow Bash + PreToolUse hook
- Why move: approving pytest ten times trains people to click yes without reading
- Diff: rm -rf, force-push, reset --hard leave deny and move into the hook
- So in Working the hook is load-bearing; missing or not executable = allowed

SLIDE 7

Talk track: hooks (the core)

- Bash(rm -rf *) misses bash -c "rm -rf tmp/" and devbox run rm -rf . (prefix match)
- PreToolUse hook reads tool_input.command, the whole string, via JSON stdin
- “A rule matches text. A hook runs a program.”
- Demo: rm -rf build/ refused → sh -c "rm -rf tmp" refused with reason text
- Point out Claude reads the reason and suggests an alternative
- If it doesn’t fire: chmod +x or the ${CLAUDE_PROJECT_DIR} path

DEMO PREP

Before the evaluation

- Scratch repo, not a client repo; launch claude inside the project
- Commit Working profile + .claude/hooks/block-destructive.sh; chmod +x
- Confirm jq is installed (hook fails open without it)
- Create build/ and tmp/ with dummy files
- Dry-run: echo '{"tool_input":{"command":"sh -c \"rm -rf tmp\""}}' | .claude/hooks/block-destructive.sh
- Run /permissions once to confirm mode and rules
- Fallback: show dry run → narrate slide 7 panels; never debug live over 30 s

PUSHBACK 1/2

Likely panel questions

- “Deny rm -rf and I’m safe?” No: rm -r -f, sh -c, devbox run all slip past
- “Is the hook bulletproof?” No: substring check; misses find -delete, shutil.rmtree
  – Defence in depth: containers, least-privilege creds, protected branches, managed policy
- “Read(.env) denied, can it cat .env?” Treat it as Read-tool only; verify on your version
- “Why not bypassPermissions?” Deny still applies; bypass is for disposable VMs only
- “Which settings win?” Managed policy > CLI flags > local > project > user

PUSHBACK 2/2

Likely panel questions

- Commit settings.local.json? No, gitignore it; commit .claude/settings.json
- JSON deny vs exit 2? Both block; JSON gives Claude a reason to act on
- Why fail open? Fail-closed blocks every command and people delete the hook
- VS Code? Rules apply everywhere; extension ignores defaultMode, set it in the UI
- Is auto safe? Probabilistic; use it on top of deny rules, never instead
- Why deny curl/wget? Wildcards can’t restrict URLs; use WebFetch domain: rules

DECK NOTES

Handle on the day (do not fix)

- Slide 2 is a placeholder: skip, introduce yourself verbally
- Slide 9 speaker notes belong to another session (SLA engine): don’t read them
- Slide 5 arrows imply a ladder: describe three modes instead
- Slide 6 code is small: name the three differences rather than reading lines
- Hands-on gap: Cautious profile has no hooks block, so sh -c test runs
  - Slide 8 blames chmod; it may be unregistered. Mention when introducing it

FACILITATION

Moves to show the panel

- Understanding checks on slides 3, 4 and 6
- Concede the hook’s limits; redirect to defence in depth
- Visible parking lot: plan mode, sandboxing, MCP permissions
- Time checks: slide 5 at 3:30, slide 7 at 6:00; apply cuts if behind